

Computação Distribuída

Processos - Parte 1

Faculdade de Computação e Informática – FCI

Universidade Presbiteriana Mackenzie

Processos

Ideia básica

Nós construímos **processadores virtuais** em software, em cima de processadores físicos:

Processador	Fornece um conjunto de instruções juntamente com a capacidade de executar automaticamente uma série dessas instruções.
Thread	Um processador de software mínimo em cujo contexto uma série de instruções pode ser executada. Salvar o contexto de uma thread implica interromper a execução atual e salvar todos os dados necessários para continuar a execução em um estágio posterior.
Processo	Um processador de software em cujo contexto uma ou mais threads podem ser executadas. Executar uma thread significa executar uma série de instruções no contexto dessa thread.

Contextos

- **Contexto do processador:** A coleção mínima de valores armazenados nos registradores de um processador usado para a execução de uma série de instruções (por exemplo, ponteiro de pilha, registradores de endereçamento, contador de programa).

Contextos

- **Contexto do processador:** A coleção mínima de valores armazenados nos registradores de um processador usado para a execução de uma série de instruções (por exemplo, ponteiro de pilha, registradores de endereçamento, contador de programa).
- **Contexto da thread:** A coleção mínima de valores armazenados em registradores e memória, usada para a execução de uma série de instruções (ou seja, contexto do processador, estado).

Contextos

- **Contexto do processador:** A coleção mínima de valores armazenados nos registradores de um processador usado para a execução de uma série de instruções (por exemplo, ponteiro de pilha, registradores de endereçamento, contador de programa).
- **Contexto da thread:** A coleção mínima de valores armazenados em registradores e memória, usada para a execução de uma série de instruções (ou seja, contexto do processador, estado).
- **Contexto do processo:** A coleção mínima de valores armazenados em registradores e memória, usada para a execução de uma thread (ou seja, contexto da thread, mas agora também pelo menos valores de registradores MMU).

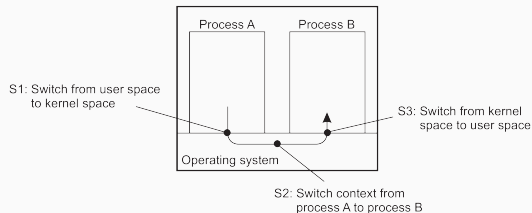
Observações

1. Threads compartilham o mesmo espaço de endereço. A troca de contexto de threads pode ser feita inteiramente independente do sistema operacional.
2. A troca de processos é geralmente (um tanto) mais cara, pois envolve colocar o SO no circuito, ou seja, fazer um *trap* para o kernel.
3. Criar e destruir threads é muito mais barato do que fazer o mesmo para processos.

Algumas razões simples

- **Evitar bloqueio desnecessário:** um processo de thread único **bloqueará** ao fazer E/S; em um processo multithread, o sistema operacional pode alternar a CPU para outra thread nesse processo.
- **Explorar paralelismo:** as threads em um processo multithread podem ser agendadas para serem executadas em paralelo em um multiprocessador ou processador multicore.
- **Evitar troca de processos:** estruturar grandes aplicações não como uma coleção de processos, mas através de múltiplas threads.

Evitar troca de contexto dispendiosa



Trade-offs

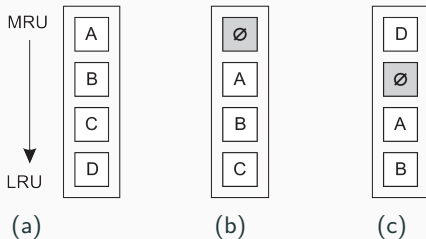
- Threads usam o mesmo espaço de endereço: mais propensas a erros
- Sem suporte do SO/HW para proteger threads usando a memória umas das outras
- A troca de contexto de threads pode ser mais rápida do que a troca de contexto de processos

O custo de uma troca de contexto

Considere um manipulador de interrupção de clock simples

- **custos diretos:** troca real e execução do código do manipulador
- **custos indiretos:** outros custos, notavelmente causados por bagunçar o cache

O que uma troca de contexto pode causar: custos indiretos



Cache

(a) antes da troca de contexto

(b) após a troca de contexto

(c) após acessar o bloco *D*.

Questão principal

Um kernel de SO deve fornecer threads ou eles devem ser implementados como pacotes de nível de usuário?

Solução no espaço do usuário

- Todas as operações podem ser completamente tratadas **dentro de um único processo** $\Rightarrow$ as implementações podem ser extremamente eficientes.
- Todos os serviços fornecidos pelo kernel são feitos **em nome do processo** no qual uma thread reside $\Rightarrow$ se o kernel decidir bloquear uma thread, todo o processo será bloqueado.
- Threads são usadas quando há muitos eventos externos: threads bloqueiam **por evento** $\Rightarrow$ se o kernel não consegue distinguir threads, como ele pode suportar o sinal de eventos para elas?

Solução no kernel

A ideia principal é que o kernel contenha a implementação de um pacote de threads. Isso significa que **todas** as operações retornam como chamadas de sistema:

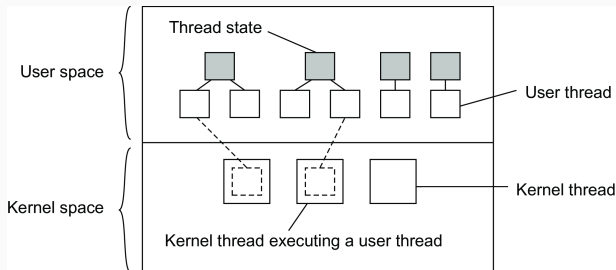
- Operações que bloqueiam uma thread não são mais um problema: o **kernel agenda outra thread disponível** dentro do mesmo processo.
- Lidar com eventos externos é simples: o **kernel** (que captura todos os eventos) **agenda a thread associada** ao evento.
- O problema é (ou costumava ser) a **perda de eficiência** porque cada operação de thread requer um *trap* para o kernel.

Conclusão – mas

- Tentar misturar threads de nível de usuário e nível de kernel em um único conceito, no entanto, o ganho de desempenho não se mostrou geralmente superior ao aumento da complexidade.

Ideia básica

Introduzir uma abordagem de threading de dois níveis: **threads de kernel** que podem executar threads de nível de usuário.



Operação principal

- Thread de usuário faz chamada de sistema $\Rightarrow$ a thread de kernel que está executando essa *thread de usuário*, bloqueia. A thread de usuário permanece **vinculada** à thread de kernel.

Operação principal

- Thread de usuário faz chamada de sistema $\Rightarrow$ a thread de kernel que está executando essa *thread de usuário*, bloqueia. A thread de usuário permanece **vinculada** à thread de kernel.
- O kernel pode agendar outra thread de kernel tendo uma thread de usuário executável **vinculada a ela**. Nota: esta thread de usuário pode mudar para **qualquer** outra thread de usuário executável atualmente no espaço do usuário.

Operação principal

- Thread de usuário faz chamada de sistema $\Rightarrow$ a thread de kernel que está executando essa *thread de usuário*, bloqueia. A thread de usuário permanece **vinculada** à thread de kernel.
- O kernel pode agendar outra thread de kernel tendo uma thread de usuário executável **vinculada a ela**. Nota: esta thread de usuário pode mudar para **qualquer** outra thread de usuário executável atualmente no espaço do usuário.
- Uma thread de usuário chama uma operação de nível de usuário bloqueante $\Rightarrow$ fazer troca de contexto para uma thread de usuário executável, (então vinculada à mesma thread de kernel).

Operação principal

- Thread de usuário faz chamada de sistema $\Rightarrow$ a thread de kernel que está executando essa *thread de usuário*, bloqueia. A thread de usuário permanece **vinculada** à thread de kernel.
- O kernel pode agendar outra thread de kernel tendo uma thread de usuário executável **vinculada a ela**. Nota: esta thread de usuário pode mudar para **qualquer** outra thread de usuário executável atualmente no espaço do usuário.
- Uma thread de usuário chama uma operação de nível de usuário bloqueante $\Rightarrow$ fazer troca de contexto para uma thread de usuário executável, (então vinculada à mesma thread de kernel).
- Quando não há threads de usuário para agendar, uma thread de kernel pode permanecer ociosa, e pode até ser removida (destruída) pelo kernel.

Cliente web multithread

Ocultando latências de rede:

- Navegador web escaneia uma página HTML recebida e descobre que mais arquivos precisam ser buscados.
- Cada arquivo é buscado por uma thread separada, cada uma fazendo uma requisição HTTP (bloqueante).
- À medida que os arquivos chegam, o navegador os exibe.

Múltiplas chamadas de solicitação-resposta para outras máquinas (RPC)

- Um cliente faz várias chamadas ao mesmo tempo, cada uma por uma thread diferente.
- Então espera até que todos os resultados tenham sido retornados.
- Nota: se as chamadas forem para servidores diferentes, podemos ter um *speed-up* linear.

Melhorar o desempenho

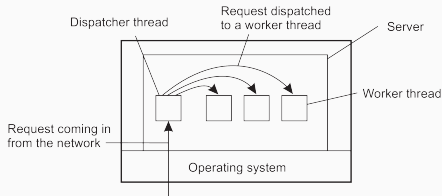
- Iniciar uma thread é mais barato do que iniciar um novo processo.
- Ter um servidor de thread único proíbe o simples *scale-up* para um sistema multiprocessador.
- Como com clientes: oculte a latência de rede reagindo à próxima requisição enquanto a anterior está sendo respondida.

Melhor estrutura

- A maioria dos servidores tem altas demandas de E/S. Usar chamadas bloqueantes simples e bem compreendidas simplifica a estrutura.
- Programas multithread tendem a ser menores e mais fáceis de entender devido ao fluxo de controle simplificado.

Por que multithreading é popular? Organização!

Modelo dispatcher/worker



Visão geral

Modelo	Características
Multithreading	Paralelismo, chamadas de sistema bloqueantes
Processo de thread único	Sem paralelismo, chamadas de sistema bloqueantes
Máquina de estados finitos	Paralelismo, chamadas de sistema não bloqueantes

Virtualização

Antes de começarmos o assunto Virtualização, vamos acertar as definições que serão utilizadas, já que há muita “confusão” no *mercado* sobre esses termos.

! Definições

Virtualização: é a aplicação de um princípio de camada com a imposição de modularidade, onde o recurso virtual exposto é idêntico ao recurso físico que está sendo virtualizado.

Máquina Virtual: é uma abstração completa de um ambiente de computação através da combinação de virtualização do processador, memória e E/S de um computador.

Hypervisor: é um software especializado que administra e roda máquinas virtuais.

Monitor de Máquina Virtual (VMM): é a parte do Hypervisor que cuida especificamente da virtualização da CPU e da memória¹.

¹os termos hypervisor e VMM são muitas vezes usados de forma intercambiável na literatura. No livro “Hardware and Software Support for Virtualization”, do qual estes slides são baseados, eles são usados com estas definições dadas aqui.

Dois princípios fundamentais em sistemas de computação

- **Camadas:** apresentação de uma abstração única, feita adicionando-se um nível de indireção quando:
 - (i) a indireção depende de uma única camada inferior;
 - (ii) usa um espaço de nomes bem definido para expor a abstração.
- **Imposição de Modularidade.** Garantia adicional de que os clientes da camada não podem dar um “bypass” na camada de abstração, por exemplo para acessar o recurso físico diretamente.

Virtualização

Portanto, não é nada mais do que uma instância de camadas para a qual a abstração exposta é equivalente ao recurso físico subjacente.

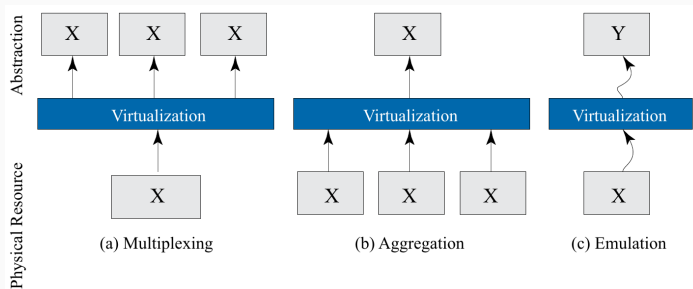
Por exemplo, pense na tecnologia RAID: grupo de discos apresentados como um disco virtual.

Virtualização

Discuta o conceito de virtualização nas seguintes situações:

1. Virtualização na Arquitetura do Computador, especialmente o conceito de memória virtual, exposto através do uso da MMU (Memory Management Unit). Como a MMU adiciona um nível de indireção, como ela impõe modularidade?
2. Virtualização dentro do Sistema Operacional, especialmente, como o SO expõe os recursos de hardware (CPU, Memória e E/S) para múltiplas aplicações concorrentes. Use os princípios fundamentais de virtualização para discutir esses conceitos.
3. Virtualização nos subsistemas de E/S, por exemplo, nos discos e controladores de discos, onde o recurso sendo virtualizado é um *array* de setores de bloco endereçáveis, como usado nos controladores RAID. Pense também em um SSD, que apesar de ser um dispositivo totalmente diferente de um disco rígido, expõe a mesma interface para o SO.

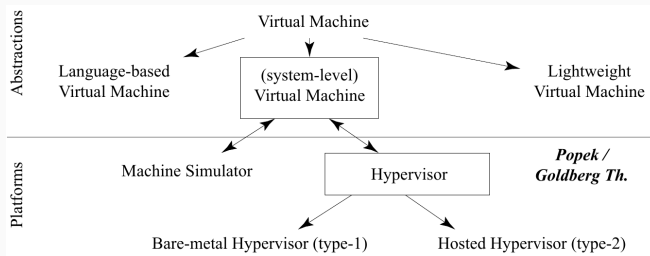
Três técnicas de virtualização



- Multiplexação – expõe um recurso entre múltiplas entidades virtuais:
 - Espaço – o recurso é particionado (em termos de espaço) entre as entidades virtuais; p.exemplo, a memória física é particionada entre diferentes espaços de endereçamento.
 - Tempo – o recurso é escalonado temporariamente entre as entidades virtuais; p.exemplo, o SO escalona a CPU entre os processos.
- Agregação – faz o oposto, pega múltiplos recursos físicos e faz com que apareçam como uma única abstração; p.exemplo, a controladora RAID agrega múltiplos discos em um único volume. O mesmo faz o controlador de memória, que agrega múltiplos pentes de memória em um único espaço de endereçamento.
- Emulação – depende de um nível de indireção em software para expor um recurso virtual ou dispositivo que corresponde ao recurso físico, mesmo se ele não estiver presente no computador atual.

! Definição

Uma **máquina virtual** é um ambiente de computação completo com seus recursos (*capabilities*) isolados de processamento, memória e canais de comunicação.



Abstrações onde se aplicam esta definição (algumas são incompatíveis entre si):

- **Máquinas virtuais baseadas em linguagens** — JVM, Microsoft Common Language Runtime, Javascript *engines*. (não estão no nosso escopo)
- **Máquinas virtuais leves** – dependem de uma combinação de mecanismos de isolamento de hardware e software para garantir que as aplicações rodem diretamente no processador de forma segura e isoladas de outros *sandboxes* e do sistema operacional subjacente; são os **Containers**; exemplo típico é o **Docker**.
- **Máquinas virtuais em nível de sistema** – na qual o ambiente de computação isolado se assemelha ao hardware de um computador, de modo que a máquina virtual pode rodar um sistema operacional padrão e suas aplicações, em completo isolamento de outras máquinas virtuais e do resto do ambiente. Estas máquinas virtuais aplicam o princípio de virtualização ao *sistema de computação completo*. Cada VM roda sua instância independente do SO, chamado de Sistema Operacional Convidado (*guest operating system*).

Categorias de Plataformas que rodam VM em nível de sistema:

- **Hypervisor** – depende da **execução direta** na CPU para eficiência máxima, idealmente para eliminar complementamente os *overheads* de desempenho. O Hypervisor permite que a VM execute suas instruções diretamente no processador. Como estas instruções operam dentro da abstração da VM, sua execução vai causar *traps*, que serão emulados pelo hypervisor. Este paradigma *trap-and-emulate* é central no projeto de hypervisors.
- **Emulador de Máquina** – tipicamente implementado como uma aplicação normal de nível de usuário, com o objetivo de prover uma simulação precisa da arquitetura virtualizada; frequentemente roda em uma fração pequena da velocidade nativa, de 5 a 1000 vezes mais lento.

Uma forma especial de software que roda máquinas virtuais com o objetivo de minimizar *overheads*.

Relacionamento entre VM e Hypervisor (Popek and Goldberg, 1974):

VM e Hypervisor

Uma VM deve ser uma duplicata eficiente e isolada da máquina real. O Hypervisor (eles chamam de VMM) é o responsável por isso: ele tem três características essenciais: primeiro, o VMM provê um ambiente para os programas que é essencialmente idêntico à máquina original; segundo, os programas rodam neste ambiente somente com uma pequena perda de desempenho; e por último, o VMM está em controle completo dos recursos do sistema.

Equivalência

A duplicação garante que o recurso exposto (i.e., a máquina virtual) é equivalente ao computador subjacente. Esse é um requisito forte que tem sido *relaxado* historicamente, quando a arquitetura exige.

Segurança (Safety)

Isolamento requer que as máquinas virtuais sejam **isoladas** umas das outras, bem como do próprio Hypervisor, que impõe modularidade ao sistema. Criticamente, a segurança do projeto é imposta pelo hypervisor sem precisar saber nada sobre o software que roda na máquina virtual, incluindo o próprio sistema operacional convidado.

Desempenho

O sistema virtual deve apresentar, no pior caso, apenas uma pequena perda de desempenho. Este requisito final separa **hypervisors** de **emuladores**. Apesar de os emuladores atenderem aos princípios de duplicação e isolamento, eles falham no critério de eficiência, mesmo emuladores que usam tradução dinâmica por hardware, pioram o desempenho do sistema alvo pelo menos em 5 vezes.

Hypervisors Tipo 1 e Tipo 2

Hypervisors podem ser classificados em duas categorias.

Hypervisors Tipo 1

Rodam diretamente no *bare metal*, ou seja, diretamente na máquina.

Hypervisors Tipo 2

Rodam em uma máquina estendida, sob o sistema operacional hospedeiro. Ele pode operar “como parte” ou “no topo” deste sistema operacional hospedeiro.

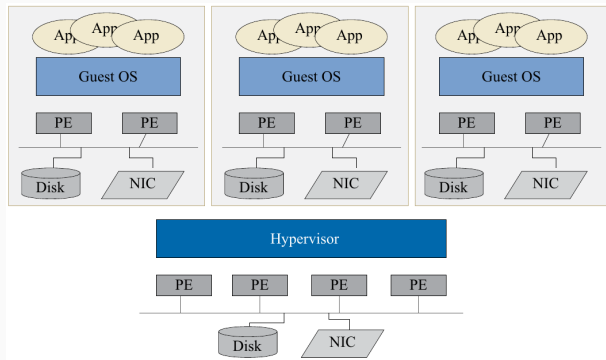
! Atenção

A ênfase aqui é sobre a alocação de recursos, não se o hypervisor roda em modo privilegiado ou não privilegiado. Em particular, um hypervisor tipo 2 pode rodar em modo kernel, e.g., Linux/KVM.

Faça uma pesquisa e classifique os seguintes Hypervisors (VMMs) em Tipo 1 ou Tipo 2, comentando aspectos históricos ou peculiares sobre eles.

- VMware ESX Server
- Xen
- Microsoft Hyper-V
- VMware Workstation
- VMware Fusion
- KVM
- Microsoft VirtualPC
- Parallels
- Oracle VirtualBox.

Esquema de um Sistema de Computação Virtualizado



PE – Processing element (Elemento de Processamento) consiste de CPU e memória física.

Esta figura representa um servidor típico; um desktop precisaria incluir teclado, mouse, portas seriais, USB, etc.

Fim

Continuaremos esse assunto de Processos, falando sobre:

- Avançando sobre o tema de Virtualização
- Containers
- Clientes e Servidores
- Migração de Código.